Ambientes de SomnGuard

SomnGuard | ADSO 3145555 | Revisión 2026-11-05

Ambientes

Estado: En progreso

Fecha de revisión: 2026-11-05

Los ambientes del sistema SomnGuard, cómo se configuran y qué reglas rigen en cada uno. Complementa a git-conventions y a local-setup.md.

Los tres ambientes

Cada ambiente es también una **rama protegida**: no se trabaja directamente sobre ella, recibe cambios por Pull Request y representa una etapa de madurez creciente.

Ambiente	Rama	Propósito	Base de datos	Migraciones
develop	`develop`	Integración del trabajo en curso	BD de desarrollo	Forward + rollback local permitido
qa	`qa`	Validación funcional y técnica (previa al release)	BD de QA	**Forward-only**
main	`main`	Producción / documentación estable	BD de producción	**Forward-only**

Promoción: `develop -> qa -> main`. El detalle del flujo de ramas (`hu-<repo>-###-<ambiente>`) está en git-conventions.md y ci-cd-strategy.md.

> **Estado verificado:** las evidencias del 2026-10-30 documentan el portal de producción (`somnguard.tech`), el health check de la API (`api.somnguard.tech/actuator/health`) y el host de descargas (`download.somnguard.tech`). Consulte el manual de despliegue y las capturas de infraestructura. Estas pruebas no verifican despliegues separados para `develop` y `qa`; no se deben inferir a partir de producción.

Configuración por ambiente - `.env.<ambiente>`

La configuración de cada ambiente vive en un archivo de entorno propio:

Archivo	Ambiente
`.env.develop`	develop
`.env.qa`	qa
`.env.main`	main

Docker Compose lee `.env` por defecto, por lo que el archivo se pasa **explícitamente** en cada comando:

```
docker compose --env-file .env.develop up postgres -d
```

Estos archivos parametrizan, como mínimo, la conexión a Postgres (host, puerto, nombre de BD, usuario, clave) del ambiente correspondiente.

Base de datos por ambiente

- Cada ambiente tiene su **propia base de datos**, con la misma estructura: **una BD** `somnguard`**, un schema por módulo** (`security`, `parameterization`, `device_management`, `telemetry_service`, `monitoring`, `analytics`). Ningún módulo escribe en `public`.

- **Datos por ambiente:**
- `develop` admite **datos de prueba** (seeds con `context`/`labels` de Liquibase).
- `qa`/`main` reciben **solo catálogos de negocio** (semillas reales, idempotentes con `ON CONFLICT DO NOTHING`).
- En `qa`/`main` las migraciones son **forward-only**; el rollback es plan de contingencia documentado, no operación rutinaria.

Advertencia de secretos

> **Regla preventiva.** Los archivos `.env.*` no deben contener contraseñas reales versionadas en Git. La auditoría de los seis repositorios y de sus referencias e historial alcanzables, realizada el 2026-11-05, encontró plantillas `.env.example`, pero no archivos `.env` reales versionados. Los valores de esas plantillas deben seguir siendo ficticios; si una credencial real aparece, hay que rotarla y revisar el historial.

Reglas:

- Versionar únicamente un `.env.example` con valores ficticios como plantilla; mantener los valores reales **fuera de git**.
- Ante una credencial que haya estado versionada: **rotar primero, avisar después** (ver política de seguridad).
- Estado objetivo: mover los secretos a un **Secret Manager** cuando exista la capa de despliegue.

Referencias

- local-setup.md - comandos por ambiente
- ci-cd-strategy.md - validación y promoción automatizadas
- git-conventions.md - ramas protegidas y flujo de promoción
- architecture-document.md - topología de despliegue por ambiente